

《软件安全》实验报告

姓名：谢小珂 学号：2310422 班级：1069

实验名称：

AFL 模糊测试实验

实验要求：

根据课本 7.4.5 章节，复现 AFL 在 KALI 下的安装、应用，查阅资料理解覆盖引导和文件变异的概念和含义。

实验过程：

1. 安装 AFL

- 在 Kali Linux 系统中，首先尝试通过 `sudo apt-get install afl` 安装 AFL，但因 AFL 已停止更新而失败。随后，通过下载源码包手动安装，如下：

```
wget http://lcamtuf.coredump.cx/afl/releases/afl-latest.tgz
```

```
(kali@kali)-[~/demo1]
$ wget http://lcamtuf.coredump.cx/afl/releases/afl-latest.tgz
URL transformed to HTTPS due to an HSTS policy
--2025-04-23 08:22:43-- https://lcamtuf.coredump.cx/afl/releases/afl-latest.tgz
Resolving lcamtuf.coredump.cx (lcamtuf.coredump.cx) ... 104.21.83.192, 172.67.180.222, 2606:4700:3036::ac43:b4de, ...
Connecting to lcamtuf.coredump.cx (lcamtuf.coredump.cx)|104.21.83.192|:443 ... connected.
HTTP request sent, awaiting response... 200 OK
Length: 835907 (816K) [application/x-gzip]
Saving to: 'afl-latest.tgz'

afl-latest.tgz 100%[=====>] 816.32K 493KB/s in 1.7s

2025-04-23 08:22:46 (493 KB/s) - 'afl-latest.tgz' saved [835907/835907]

(kali@kali)-[~/demo1]
$
```

- 下载之后利用 `tar xvf afl-latest.tgz` 解压，如图：

```
(kali@kali)-[~/demo1]
$ tar xvf afl-latest.tgz
afl-2.52b/
afl-2.52b/afl-as.h
afl-2.52b/libtokencap/
afl-2.52b/libtokencap/libtokencap.so.c
afl-2.52b/libtokencap/README.tokencap
afl-2.52b/libtokencap/Makefile
afl-2.52b/alloc-inl.h
afl-2.52b/config.h
afl-2.52b/test-instr.c
afl-2.52b/afl-analyze.c
afl-2.52b/README
afl-2.52b/afl-showmap.c
afl-2.52b/experimental/
afl-2.52b/experimental/clang_asm_normalize/
afl-2.52b/experimental/clang_asm_normalize/as
afl-2.52b/experimental/README.experiments
afl-2.52b/experimental/distributed_fuzzing/
afl-2.52b/experimental/distributed_fuzzing/sync_script.sh
afl-2.52b/experimental/crash_triage/
afl-2.52b/experimental/crash_triage/triage_crashes.sh
afl-2.52b/experimental/libpng_no_checksum/
afl-2.52b/experimental/libpng_no_checksum/libpng-nocrc.patch
afl-2.52b/experimental/bash_shellshock/
afl-2.52b/experimental/bash_shellshock/shellshock-fuzz.diff
afl-2.52b/experimental/canvas_harness/
afl-2.52b/experimental/canvas_harness/canvas_harness.html
afl-2.52b/experimental/asan_cgroups/
afl-2.52b/experimental/asan_cgroups/limit_memory.sh
afl-2.52b/experimental/post_library/
afl-2.52b/experimental/post_library/post_library.so.c
afl-2.52b/experimental/post_library/post_library_png.so.c
afl-2.52b/experimental/persistent_demo/
```

- 解压后通过 make 编译，make install 安装

```
(kali㉿kali)-[~/demo1]
$ cd afl-2.52b

(kali㉿kali)-[~/demo1/afl-2.52b]
$ ls
afl-analyze.c  afl-plot      dictionaries  Makefile
afl-as.c       afl-showmap.c docs          qemu_mode
afl-as.h       afl-tmin.c    experimental  QuickStartGuide.txt
afl-cmin       afl-whatsup   hash.h        README
afl-fuzz.c     alloc-inl.h  libdislocator testcases
afl-gcc.c      config.h     libtokencap   test-instr.c
afl-gotcpu.c   debug.h      llvm_mode     types.h

(kali㉿kali)-[~/demo1/afl-2.52b]
$ sudo make 66sudo make install
[sudo] password for kali:
[*] Checking for the ability to compile x86 code...
[+] Everything seems to be working, ready to compile.
cc -O3 -funroll-loops -Wall -D_FORTIFY_SOURCE=2 -g -Wno-pointer-sign -DAFL_PATH="/usr/local/lib/afl/" -DDOC_PATH="/usr/local/share/doc/afl/" -DBIN_PATH="/usr/local/bin/" afl-gcc.c -o afl-gcc -ldl
set -e; for i in afl-g++ afl-clang afl-clang++; do ln -sf afl-gcc $i; done
cc -O3 -funroll-loops -Wall -D_FORTIFY_SOURCE=2 -g -Wno-pointer-sign -DAFL_PATH="/usr/local/lib/afl/" -DDOC_PATH="/usr/local/share/doc/afl/" -DBIN_PATH="/usr/local/bin/" afl-fuzz.c -o afl-fuzz -ldl
cc -O3 -funroll-loops -Wall -D_FORTIFY_SOURCE=2 -g -Wno-pointer-sign -DAFL_PATH="/usr/local/lib/afl/" -DDOC_PATH="/usr/local/share/doc/afl/" -DBIN_PATH="/usr/local/bin/" afl-showmap.c -o afl-showmap -ldl
cc -O3 -funroll-loops -Wall -D_FORTIFY_SOURCE=2 -g -Wno-pointer-sign -DAFL_PATH="/usr/local/lib/afl/" -DDOC_PATH="/usr/local/share/doc/afl/" -DBIN_PATH="/usr/local/bin/" afl-tmin.c -o afl-tmin -ldl
cc -O3 -funroll-loops -Wall -D_FORTIFY_SOURCE=2 -g -Wno-pointer-sign -DAFL_PATH="/usr/local/lib/afl/" -DDOC_PATH="/usr/local/share/doc/afl/" -DBIN_PATH="/usr/local/bin/" afl-gotcpu.c -o afl-gotcpu -ldl
cc -O3 -funroll-loops -Wall -D_FORTIFY_SOURCE=2 -g -Wno-pointer-sign -DAFL_PATH="/usr/local/lib/afl/" -DDOC_PATH="/usr/local/share/doc/afl/" -DBIN_PATH="/usr/local/bin/" afl-gotcpu.c -o afl-gotcpu -ldl
```

- 验证安装：检查/usr/local/bin 目录下的 AFL 可执行文件。

```
if [ -f afl-qemu-trace ]; then install -m 755 afl-qemu-trace ${DESTDIR}/usr/local/bin; fi
if [ -f afl-clang-fast -a -f afl-llvm-pass.so -a -f afl-llvm-rt.o ]; then set -e; install -m 755 afl-clang-fast ${DESTDIR}/usr/local/bin; ln -sf afl-clang-fast ${DESTDIR}/usr/local/bin/afl-clang-fast++; install -m 755 afl-llvm-pass.so afl-llvm-rt.o ${DESTDIR}/usr/local/lib/afl; fi
if [ -f afl-llvm-rt-32.o ]; then set -e; install -m 755 afl-llvm-rt-32.o ${DESTDIR}/usr/local/lib/afl; fi
if [ -f afl-llvm-rt-64.o ]; then set -e; install -m 755 afl-llvm-rt-64.o ${DESTDIR}/usr/local/lib/afl; fi
set -e; for i in afl-g++ afl-clang afl-clang++; do ln -sf afl-gcc ${DESTDIR}/usr/local/bin/$i; done
install -m 755 afl-as ${DESTDIR}/usr/local/lib/afl
ln -sf afl-as ${DESTDIR}/usr/local/lib/afl/as
install -m 644 docs/README docs/ChangeLog docs/*.txt ${DESTDIR}/usr/local/share/doc/afl
cp -r testcases/ ${DESTDIR}/usr/local/share/afl
cp -r dictionaries/ ${DESTDIR}/usr/local/share/afl

(kali㉿kali)-[~/demo1/afl-2.52b]
$ ls /usr/local/bin
afl-analyze  afl-clang++  afl-fuzz  afl-gcc  afl-plot  afl-tmin
afl-clang    afl-cmin     afl-g++   afl-gotcpu  afl-showmap  afl-whatsup

(kali㉿kali)-[~/demo1/afl-2.52b]
$ ls /usr/local/bin/afl*
/usr/local/bin/afl-analyze  /usr/local/bin/afl-gcc
/usr/local/bin/afl-clang    /usr/local/bin/afl-gotcpu
/usr/local/bin/afl-clang++  /usr/local/bin/afl-plot
/usr/local/bin/afl-cmin     /usr/local/bin/afl-showmap
/usr/local/bin/afl-fuzz     /usr/local/bin/afl-tmin
/usr/local/bin/afl-g++      /usr/local/bin/afl-whatsup

(kali㉿kali)-[~/demo1/afl-2.52b]
$
```

2. 准备测试程序

- 在 demo1 文件夹中创建 C 程序文件 test.c，代码功能为：若输入字符串为“deadbeef”则触发程序终止（abort()），否则原样输出。代码如下：

- 关键代码:

```
if (ptr[0] == 'd' && ptr[1] == 'e' && ... && ptr[7] == 'f') abort();
```

```
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char **argv){
    char ptr[20];
    if(argc>1){
        FILE *fp = fopen(argv[1],"r");
        fgets(ptr,sizeof(ptr),fp);
    }
    else{
        fgets(ptr, sizeof(ptr), stdin);
    }
    printf("%s",ptr);
    if(ptr[0]=='d'){
        if(ptr[1]=='e'){
            if(ptr[2]=='a'){
                if(ptr[3]=='d'){
                    if(ptr[4]=='b'){
                        if(ptr[5]=='e'){
                            if(ptr[6]=='e'){
                                if(ptr[7]=='f'){
                                    abort();
                                }
                            }
                        }
                    }
                }
            }
        }
        else printf("%c",ptr[7]);
    }
    else printf("%c",ptr[6]);
    }
    else printf("%c",ptr[5]);
    }
    else printf("%c",ptr[4]);
    }
}
```

- 使用 AFL 的编译器包装器 afl-gcc 编译程序:

```
afl-gcc -o test test.c
```

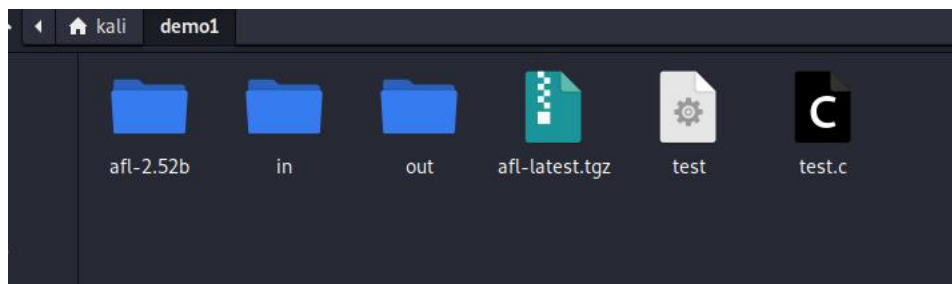
```
(kali@kali)~/demo1
$ afl-gcc -o test test.c
afl-cc 2.52b by <lcamtuf@google.com>
afl-as 2.52b by <lcamtuf@google.com>
[+] Instrumented 14 locations (64-bit, non-hardened mode, ratio 100%).
```

- 通过 readelf -s ./test | grep afl 验证插桩成功 (AFL 的符号存在)。

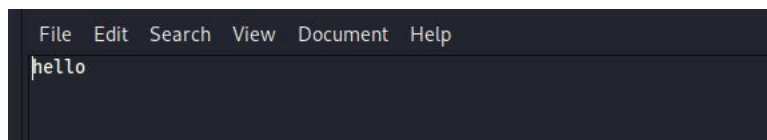
```
(kali@kali)~/demo1
$ readelf -s ./test | grep afl
35: 00000000000001628 0 NOTYPE LOCAL DEFAULT 14 __afl_maybe_lo
37: 000000000000040b0 8 OBJECT LOCAL DEFAULT 25 __afl_area_ptr
38: 00000000000001658 0 NOTYPE LOCAL DEFAULT 14 __afl_setup
39: 00000000000001638 0 NOTYPE LOCAL DEFAULT 14 __afl_store
40: 000000000000040b8 8 OBJECT LOCAL DEFAULT 25 __afl_prev_loc
41: 00000000000001650 0 NOTYPE LOCAL DEFAULT 14 __afl_return_fa
42: 000000000000040c8 1 OBJECT LOCAL DEFAULT 25 __afl_setup_fa
43: 00000000000001679 0 NOTYPE LOCAL DEFAULT 14 __afl_setup_fi
45: 0000000000000193e 0 NOTYPE LOCAL DEFAULT 14 __afl_setup_ab
46: 00000000000001793 0 NOTYPE LOCAL DEFAULT 14 __afl_forkserv
47: 000000000000040c4 4 OBJECT LOCAL DEFAULT 25 __afl_temp
48: 00000000000001851 0 NOTYPE LOCAL DEFAULT 14 __afl_fork_res
49: 000000000000017b9 0 NOTYPE LOCAL DEFAULT 14 __afl_fork_wai
50: 00000000000001936 0 NOTYPE LOCAL DEFAULT 14 __afl_die
51: 000000000000040c0 4 OBJECT LOCAL DEFAULT 25 __afl_fork_pid
98: 000000000000040d0 8 OBJECT GLOBAL DEFAULT 25 __afl_global_a
```

3.配置测试环境

- 创建输入 (in) 和输出 (out) 文件夹, 在 in 中生成初始测试用例。



通过命令 `echo hello> in/foo` 在 in 文件夹中写入 hello 。

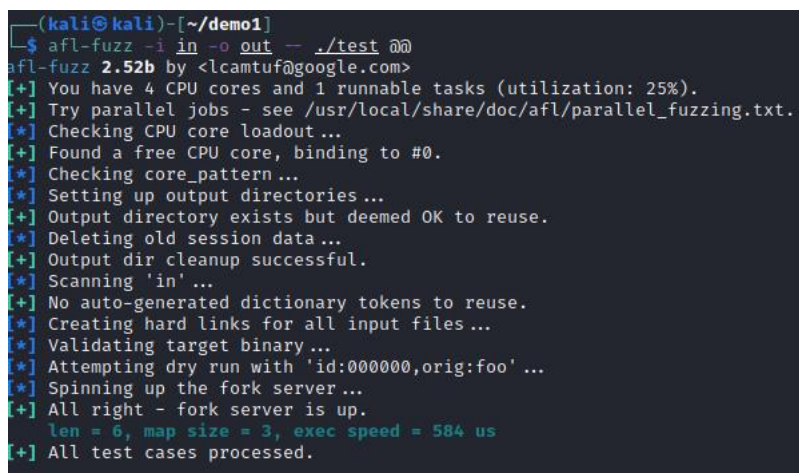


4.运行模糊测试

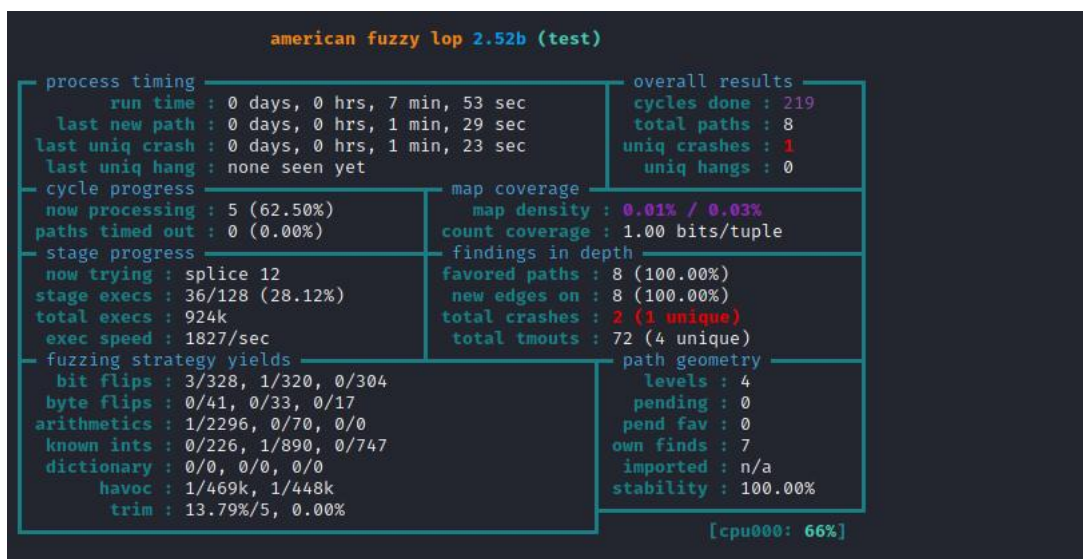
- 启动 AFL 模糊测试命令：

```
afl-fuzz -i in -o out -- ./test @@
```

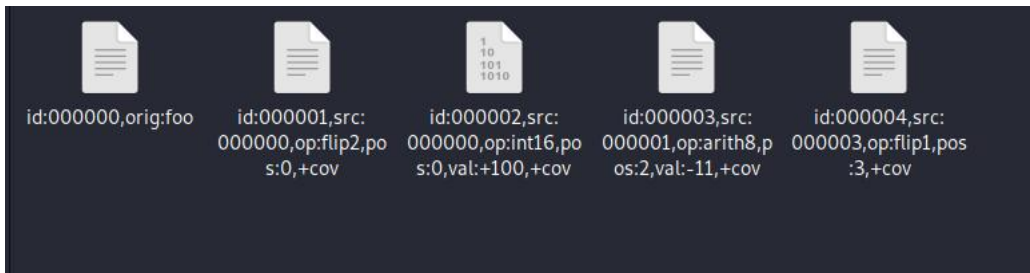
@@表示 AFL 将自动替换为生成的输入文件。



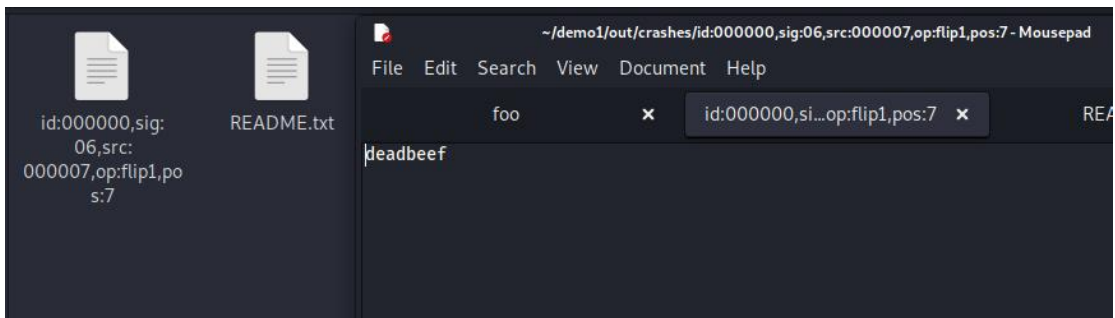
• AFL 界面显示覆盖率增长和崩溃情况。当 total crashes 非零时，表示触发了程序异常（如 “deadbeef” 导致 abort()）。



- AFL 模糊测试过程中，工具自动生成的变异测试用例会实时记录在队列中。



- 在测试结果目录的 `crashes` 文件夹中，通过查看错误日志可以确认，触发程序崩溃的测试输入正是预设的字符串 "deadbeef"。



心得体会：

一、覆盖引导与文件变异的理解

1. 覆盖引导

覆盖引导是 AFL 等现代模糊测试工具的核心机制，其通过实时监控程序的执行路径覆盖率来指导测试用例的生成。具体表现为：通过插桩（Instrumentation）技术，在程序关键分支点插入统计代码，记录每个测试用例触发的代码路径。同时，优先保留那些触发新路径的测试用例，通过遗传算法不断优化测试输入。相比盲目随机测试，覆盖引导能快速探索程序的深层逻辑分支。

在测试程序 `test.c` 中，AFL 通过插桩发现输入 `deadbeef` 会触发独特的崩溃路径（`abort()`），并将此输入保存为高优先级用例，体现了覆盖引导的精准性。

2. 文件变异

文件变异是 AFL 生成测试用例的主要手段，通过对初始输入进行结构化变异，生成大量衍生用例。其关键策略包括以下几点。如位翻转（Bit Flipping）：随机修改输入文件的比特位；块操作（Block Operations）：插入、删除或替换数据块；算术变异：对数值型数据进行加减运算；字典变异：结合预定义的敏感词生成针对性用例。

实验中，AFL 从初始输入 `hello` 出发，通过变异逐渐生成接近 `deadbeef` 的字符串，最终触发程序崩溃。此过程展示了变异策略如何系统性逼近漏洞触发条件。

二、实验总结

1. 成功在 Kali Linux 系统上通过源码编译安装了 AFL 模糊测试工具，解决了 apt 安装不可用的问题。
2. 编写了具有特定触发条件（输入 `deadbeef` 时 `abort`）的测试程序 `test.c`，并使用 `afl-gcc` 完成编译和插桩。

3. 建立了完整的测试环境，包括输入输出目录结构、初始测试用例配置等。
4. 运行 afl-fuzz 进行模糊测试，成功触发了预设的程序崩溃条件。
5. 通过分析 crashes 目录中的错误日志，验证了测试结果的有效性。

三、技术收获

1. 掌握了 AFL 工具的基本使用方法，包括安装配置、测试程序准备、测试执行等全流程。
2. 深入理解了覆盖引导模糊测试的工作原理，包括路径覆盖监控、测试用例智能变异等核心机制。
3. 实践了插桩技术的实际应用，了解了其在提高测试效率方面的重要作用。
4. 学会了如何分析测试结果，定位程序漏洞。

四、改进方向

1. 尝试更复杂的测试用例设计，提高测试深度。
2. 探索 AFL 的更多高级功能，如并行测试、字典测试等。
3. 结合其他安全测试工具，构建更完善的测试方案。
4. 深入研究插桩技术的实现原理，尝试自定义插桩策略。

通过本次 AFL 模糊测试实验，我深入理解了覆盖引导和文件变异在软件安全测试中的重要性。AFL 通过动态监控程序执行路径，智能生成测试用例，有效提高了漏洞发现的效率。实验中，当输入字符串为预设的 deadbeef 时，程序按预期触发崩溃，验证了 AFL 在检测边界条件漏洞方面的强大能力。

此外，手动编译安装 AFL 的过程让我熟悉了 Linux 环境下工具链的配置，而插桩技术的应用则让我认识到代码覆盖率对安全测试的关键作用。通过分析 crashes 目录中的错误日志，我进一步掌握了如何定位和复现程序异常，这对今后的漏洞分析和修复工作具有重要指导意义。

总结来说，本次实验不仅巩固了我的理论知识，还提升了实践能力，让我对自动化模糊测试的价值有了更深刻的认识。未来，我将探索更多变异策略和工具优化方法，以进一步提升测试的全面性和效率。